

# RANGKUMAN SUPER LENGKAP ADMINISTRASI BASIS DATA

*Materi 7 · Materi 8 · Materi 9*

 **Materi 7: Strategi Backup Data**

 **Materi 8: Transaction Log Backup & Point-in-Time Recovery**

 **Materi 9: Pemeliharaan Database (Maintenance)**



## MATERI 7: STRATEGI BACKUP DATA

Backup adalah proses menyalin data database ke tempat lain agar bisa dikembalikan (restore) jika terjadi kerusakan atau kesalahan. Pada SQL Server, ada tiga jenis backup utama yang perlu kamu kuasai.

### 1 Recovery Model — Pengaturan Awal Wajib

Sebelum bisa melakukan Full Backup atau apapun, kamu HARUS mengatur Recovery Model database. Bayangkan Recovery Model sebagai 'mode rekam' database kamu — ia menentukan seberapa detail SQL Server mencatat perubahan data.

| Recovery Model     | Penjelasan                                                                                                                                                           |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SIMPLE</b>      | Hanya mencatat transaksi seperlunya. TIDAK bisa backup transaction log. Cocok untuk database non-kritis.                                                             |
| <b>FULL</b>        | Mencatat SEMUA transaksi secara detail. Wajib dipakai jika ingin backup log & point-in-time recovery. <input checked="" type="checkbox"/> WAJIB untuk praktikum ini. |
| <b>BULK_LOGGED</b> | Mirip FULL, tapi transaksi massal (seperti import data) dicatat lebih ringkas.                                                                                       |

#### ⚠ ATURAN PENTING Recovery Model

Setelah mengubah ke FULL → WAJIB langsung lakukan Full Backup.  
Jika tidak, SQL Server tidak bisa melakukan Log Backup.  
Log Backup HANYA bisa dilakukan setelah ada Full Backup pertama.

#### Cara Cek Recovery Model:

```
SELECT name, recovery_model_desc
FROM sys.databases
WHERE name = 'DBKampus';
```

#### Cara Ubah ke FULL Recovery:

```
ALTER DATABASE DBKampus
SET RECOVERY FULL;
```

### 2 Full Backup — Cadangan Lengkap Seluruh Database

**Full Backup** = menyalin SEMUA data database pada saat backup dijalankan. Ibarat memotret seluruh database dalam satu foto. Ini menjadi **fondasi/dasar** untuk semua jenis backup lainnya.

#### 📌 Analogi Mudah

Bayangkan kamu punya buku catatan 100 halaman.  
Full Backup = Fotokopi semua 100 halaman. Simpan di laci.  
Besoknya kamu tambah 10 halaman baru → butuh Differential Backup.

## Perintah Full Backup:

```
BACKUP DATABASE DBKampus
TO DISK = 'D:\BackupSQL\DBKampus_Full.bak'
WITH FORMAT,
    INIT,
    NAME = 'Full Backup DBKampus',
    STATS = 10;
```

| Opsi            | Arti                                                         |
|-----------------|--------------------------------------------------------------|
| <b>FORMAT</b>   | Menginisialisasi file backup baru (hapus isi lama jika ada). |
| <b>INIT</b>     | Menimpa media backup yang sudah ada.                         |
| <b>NAME</b>     | Label/nama deskriptif untuk backup set ini.                  |
| <b>STATS=10</b> | Menampilkan progress setiap 10%.                             |

## 3 Differential Backup — Cadangan Perubahan Sejak Full Terakhir

**Differential Backup** = hanya menyimpan data yang **BERUBAH** sejak Full Backup terakhir. Lebih kecil dan lebih cepat daripada Full Backup.



### Analogi Lanjutan

Full Backup → 100 halaman difotokopi.

Differential → Hanya 10 halaman baru yang difotokopi (tidak perlu fotokopi ulang 100 halaman lama).

Kalau besoknya tambah 5 halaman lagi → Differential ke-2 = 15 halaman (sejak Full terakhir).

## Perintah Differential Backup:

```
BACKUP DATABASE DBKampus
TO DISK = 'D:\BackupSQL\DBKampus_Diff.bak'
WITH DIFFERENTIAL,
    INIT,
    NAME = 'Differential Backup DBKampus',
    STATS = 10;
```

## 4 Proses Restore — Mengembalikan Database

Ketika terjadi bencana data (human error, korupsi data, dll.), kamu perlu melakukan restore. Urutan restore SANGAT penting dan tidak boleh dibalik.



### URUTAN RESTORE WAJIB (Jangan Sampai Terbalik!)

Langkah 1: Set database ke Single User Mode (agar tidak ada yang mengakses).

Langkah 2: Restore Full Backup → gunakan WITH NORECOVERY (database belum aktif).

Langkah 3: Restore Differential Backup → gunakan WITH RECOVERY (database aktif kembali).

Langkah 4: Set kembali ke Multi User Mode.

### Step 1 — Single User Mode:

```
USE master;  
ALTER DATABASE DBKampus  
SET SINGLE_USER  
WITH ROLLBACK IMMEDIATE;
```

### Step 2 — Restore Full Backup (NORECOVERY):

```
RESTORE DATABASE DBKampus  
FROM DISK = 'D:\BackupSQL\DBKampus_Full.bak'  
WITH NORECOVERY,  
     REPLACE,  
     STATS = 10;
```

### Step 3 — Restore Differential Backup (RECOVERY):

```
RESTORE DATABASE DBKampus  
FROM DISK = 'D:\BackupSQL\DBKampus_Diff.bak'  
WITH RECOVERY,  
     STATS = 10;
```

### Step 4 — Kembali ke Multi User:

```
ALTER DATABASE DBKampus  
SET MULTI_USER;
```

## 5 Analisis Pertanyaan Ujian — Jawab dengan Yakin!

### ? Q: Mengapa Full Backup harus di-restore dulu?

Jawab: Karena Differential Backup TIDAK berisi semua data. Ia hanya berisi perubahan sejak Full Backup.

Full Backup adalah fondasi. Tanpanya, Differential Backup tidak bisa berdiri sendiri.

Analoginya: kamu tidak bisa menempel foto tambahan tanpa album dasarnya.

### ? Q: Apa yang terjadi jika langsung restore Differential?

Jawab: GAGAL / ERROR. SQL Server akan menolak restore Differential karena tidak ada 'base' (dasar) yang valid dari Full Backup.

SQL Server perlu tahu 'kondisi awal' sebelum menerapkan perubahan.

**? Q: Mengapa NORECOVERY digunakan pada Full Backup?**

Jawab: Karena setelah Full Backup di-restore, kita belum selesai.

Masih ada Differential (atau Log) yang harus di-apply berikutnya.

NORECOVERY = 'tunggu dulu, saya masih mau tambahkan backup lain'.

RECOVERY = 'selesai, database bisa dipakai sekarang'.

**? Q: Apa fungsi REPLACE?**

Jawab: Memberitahu SQL Server untuk menimpa/menggantikan database yang sudah ada.

Tanpa REPLACE, jika database DBKampus sudah ada, restore akan gagal.

REPLACE memaksa SQL Server menggantikan database lama dengan backup.

## MATERI 8: TRANSACTION LOG BACKUP & POINT-IN-TIME RECOVERY

*Transaction Log Backup adalah tingkatan backup yang paling detail. Jika Full & Differential backup seperti 'foto database', Transaction Log backup seperti 'rekaman video' setiap perubahan yang terjadi.*

### 1 Apa itu Transaction Log?

Transaction Log adalah file (.ldf) yang mencatat SETIAP operasi yang terjadi di database: setiap INSERT, UPDATE, DELETE, CREATE TABLE — semua tercatat dengan timestamp (waktu tepat) dan urutan transaksinya.

#### Analogi CCTV

Full Backup = Foto kondisi database kemarin pagi.

Differential = Foto kondisi database siang tadi.

Transaction Log = Rekaman CCTV menit demi menit sepanjang hari.

Dengan rekaman CCTV, kamu bisa rewind ke detik TEPAT sebelum kesalahan terjadi!

### 2 Alur Lengkap Transaction Log Backup

| Langkah   | Aksi                                                    |
|-----------|---------------------------------------------------------|
| Langkah 1 | Pastikan Recovery Model = FULL                          |
| Langkah 2 | Lakukan Full Backup (WAJIB sebagai dasar log)           |
| Langkah 3 | Lakukan transaksi/perubahan data (INSERT, UPDATE, dll.) |
| Langkah 4 | Lakukan Log Backup Pertama (Log1.trn)                   |
| Langkah 5 | Lakukan transaksi/perubahan data lagi                   |
| Langkah 6 | Lakukan Log Backup Kedua (Log2.trn)                     |
| Langkah 7 | Simulasi Human Error (DELETE FROM ...)                  |
| Langkah 8 | Restore Point-in-Time (kembalikan sebelum DELETE)       |

### 3 Perintah Transaction Log Backup

**Log Backup Pertama (setelah transaksi pertama):**

```
BACKUP LOG DBKampus
TO DISK = 'D:\BackupSQL\DBKampus_Log1.trn'
WITH INIT,
STATS = 10;
```

**Log Backup Kedua (setelah transaksi kedua):**

```
BACKUP LOG DBKampus
```

```
TO DISK = 'D:\BackupSQL\DBKampus_Log2.trn'
WITH INIT,
    STATS = 10;
```

### Kunci Memahami Log Backup

Log1.trn berisi: semua perubahan dari Full Backup s/d Log1 dibuat.

Log2.trn berisi: semua perubahan dari Log1 s/d Log2 dibuat.

Setiap log backup TIDAK overlap — mereka berurutan seperti rantai.

Jika salah satu file .trn hilang, rantai restore PUTUS dan tidak bisa dilanjutkan.

## Point-in-Time Recovery — Kembali ke Waktu Tertentu

**Point-in-Time Recovery** = kemampuan mengembalikan database ke kondisi pada **waktu spesifik (jam:menit:detik)** — bukan hanya ke waktu backup terakhir.

### Skenario Praktikum

Situasi: Ada data penting. Lalu seseorang tidak sengaja menghapus semua data (DELETE FROM) pukul 06:10.

Tujuan: Kembalikan database ke kondisi SEBELUM pukul 06:10.

Solusi: Restore Full → Restore Log1 → Restore Log2 WITH STOPAT '06:09:59'

## Urutan Restore Point-in-Time (3 langkah kunci):

### Step 1 — Restore Full Backup (NORECOVERY):

```
/* Di database master */
RESTORE DATABASE DBKampus
FROM DISK = 'D:\BackupSQL\DBKampus_Full2.bak'
WITH NORECOVERY,
    REPLACE;
```

### Step 2 — Restore Log1 (NORECOVERY):

```
RESTORE LOG DBKampus
FROM DISK = 'D:\BackupSQL\DBKampus_Log1.trn'
WITH NORECOVERY;
```

### Step 3 — Restore Log2 dengan STOPAT (RECOVERY):

```
RESTORE LOG DBKampus
FROM DISK = 'D:\BackupSQL\DBKampus_Log2.trn'
WITH STOPAT = '2026-05-05 06:10:00',
    RECOVERY;
```

### STOPAT — Kunci Point-in-Time Recovery

STOPAT = titik waktu tujuan restore.

SQL Server akan menerapkan log sampai SEBELUM waktu tersebut.

Contoh: DELETE terjadi pukul 06:04, maka STOPAT = '06:03:59' (sebelum DELETE).

Setelah STOPAT, gunakan WITH RECOVERY agar database aktif kembali.

## 5 Troubleshooting — Masalah Umum

| Masalah                                             | Solusi                                                                                                                                |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Database stuck di RESTORING</b>                  | Terjadi karena restore terakhir memakai WITH NORECOVERY tapi belum ditutup. Solusi: Jalankan RESTORE DATABASE DBKampus WITH RECOVERY; |
| <b>Error: Log tail backup required</b>              | SQL Server meminta backup tail-of-log sebelum restore. Solusi: Gunakan WITH REPLACE pada restore pertama.                             |
| <b>Query tidak menampilkan data setelah restore</b> | Pastikan sudah menjalankan RESTORE DATABASE DBKampus WITH RECOVERY; dan cek status database.                                          |
| <b>Progress restore tidak muncul</b>                | Gunakan: SELECT percent_complete FROM sys.dm_exec_requests WHERE command LIKE 'RESTORE%'                                              |

### Cek Status Database:

```
SELECT name, state_desc
FROM sys.databases
WHERE name = 'DBKampus';

-- Hasil yang diharapkan setelah restore selesai: ONLINE
-- Jika masih: RESTORING → belum selesai atau perlu WITH RECOVERY
```

### Cek Progress Restore (Jika Sedang Berjalan):

```
SELECT percent_complete,
       start_time,
       estimated_completion_time/1000 AS estimasi_detik,
       command
FROM sys.dm_exec_requests
WHERE command LIKE 'RESTORE%';
```

## 6 Perbandingan Tiga Jenis Backup — Ringkasan

| Jenis                  | Cakupan             | Isi              | Ukuran | Kecepatan | Restore                 |
|------------------------|---------------------|------------------|--------|-----------|-------------------------|
| <b>Full Backup</b>     | Seluruh database    | Semua data       | Besar  | Lambat    | Bisa (langsung)         |
| <b>Differential</b>    | Sejak Full terakhir | Perubahan saja   | Sedang | Sedang    | Perlu Full dulu         |
| <b>Transaction Log</b> | Sejak Log terakhir  | Setiap transaksi | Kecil  | Cepat     | Perlu Full + urutan Log |



## MATERI 9: PEMELIHARAAN DATABASE (MAINTENANCE)

*Maintenance database adalah kegiatan rutin untuk menjaga database tetap sehat, cepat, dan bebas korupsi data. Seperti servis kendaraan secara berkala agar tidak mogok.*

### 1 Fragmentasi Index — Apa dan Mengapa Berbahaya?

**Index** adalah struktur data yang membantu SQL Server menemukan baris data dengan cepat (seperti daftar isi buku). **Fragmentasi** terjadi ketika halaman-halaman index tidak teratur/berurutan lagi, sehingga SQL Server harus 'melompat-lompat' membaca data → performa turun.

#### Analogi Fragmentasi

Bayangkan kamu punya buku resep 100 halaman yang SANGAT teratur. Lalu kamu: robek beberapa halaman, selipkan halaman baru di tempat acak, coret beberapa entri. Sekarang mencari resep jadi lebih susah → itulah fragmentasi! Fragmentasi tinggi = performa query turun drastis.

#### Cek Tingkat Fragmentasi:

```
SELECT
    OBJECT_NAME(object_id) AS NamaTabel,
    avg_fragmentation_in_percent,
    page_count
FROM sys.dm_db_index_physical_stats
    (DB_ID(), OBJECT_ID('akademik.Mahasiswa'), NULL, NULL, NULL);
```

| Tingkat Fragmentasi | Status | Tindakan                                                |
|---------------------|--------|---------------------------------------------------------|
| 0% – 5%             | Normal | Tidak perlu tindakan apapun.                            |
| 5% – 30%            | Sedang | Lakukan REORGANIZE → lebih ringan, tidak lock database. |
| > 30%               | Parah  | Lakukan REBUILD → lebih kuat, tapi lebih lama & lock.   |

### 2 REORGANIZE — Merapikan Index Secara Online

**REORGANIZE** = merapikan halaman-halaman index yang tidak teratur. Prosesnya **online** (database tetap bisa diakses user lain selama proses berjalan). Cocok untuk fragmentasi 5–30%.

#### Perintah REORGANIZE:

```
ALTER INDEX ALL ON akademik.Mahasiswa
REORGANIZE;
```

#### Cek fragmentasi setelah REORGANIZE (harusnya turun):

```
SELECT avg_fragmentation_in_percent, page_count
FROM sys.dm_db_index_physical_stats
(DB_ID(), OBJECT_ID('akademik.Mahasiswa'), NULL, NULL, NULL);
```

### 3 REBUILD — Bangun Ulang Index dari Nol

**REBUILD** = menghapus seluruh index lama dan membangun ulang dari scratch. Hasilnya fragmentasi hampir 0%. Cocok untuk fragmentasi > 30%. Prosesnya **lebih lama dan mengunci database (offline) selama proses** (di edisi Standard).

#### Perintah REBUILD:

```
ALTER INDEX ALL ON akademik.Mahasiswa
REBUILD;
```

#### REORGANIZE vs REBUILD — Kapan Pakai Mana?

REORGANIZE: Fragmentasi 5-30%, database sibuk, tidak mau lock terlalu lama.

REBUILD: Fragmentasi > 30%, bisa dilakukan di luar jam sibuk (malam/weekend).

Tips: Setelah REBUILD, selalu UPDATE STATISTICS agar query optimizer pakai data terbaru.

### 4 UPDATE STATISTICS — Perbarui Informasi Distribusi Data

**Statistics** adalah informasi tentang distribusi nilai data dalam tabel (misal: berapa banyak mahasiswa dari Teknik Informatika?). SQL Server Query Optimizer menggunakan statistics untuk memilih **plan eksekusi query yang paling efisien**. Jika statistics usang → query bisa sangat lambat!

#### Analogi Statistics

Statistics = sensus penduduk kota.

Query Optimizer = GPS navigasi yang butuh data populasi untuk cari rute terbaik.

Jika data sensus ketinggalan 5 tahun → GPS kasih rute yang sudah tidak optimal.

UPDATE STATISTICS = perbarui data sensus agar GPS kasih rute terbaik!

#### Cek Nama Statistics:

```
SELECT name
FROM sys.stats
WHERE object_id = OBJECT_ID('akademik.Mahasiswa');
```

#### Update Statistics Satu Tabel:

```
UPDATE STATISTICS akademik.Mahasiswa;
```

#### Update Statistics Seluruh Database (Semua Tabel Sekaligus):

```
EXEC sp_updatestats;
```

## 5 DBCC CHECKDB — Pemeriksaan Integritas Database

**DBCC CHECKDB** adalah perintah untuk memeriksa integritas fisik dan logis seluruh database. Ia memeriksa: apakah ada halaman data yang rusak, apakah index konsisten dengan data, apakah ada korupsi data.



### Analogi Medical Check-Up

DBCC CHECKDB = medical check-up lengkap untuk database.  
Ia memeriksa 'organ-organ' database (tabel, index, halaman data).  
Jika tidak ada error → database sehat.  
Jika ada error → segera tangani sebelum data rusak parah!

### Perintah DBCC CHECKDB:

```
DBCC CHECKDB ('DBKampus') WITH NO_INFOMSGS;
```

| Opsi/Hasil                                     | Penjelasan                                                       |
|------------------------------------------------|------------------------------------------------------------------|
| <b>WITH NO_INFOMSGS</b>                        | Hanya tampilkan error/warning, bukan semua pesan informasi.      |
| <b>Hasil: Commands completed successfully.</b> | Database sehat! Tidak ada korupsi data.                          |
| <b>Hasil: ada error message</b>                | Ada masalah → perlu tindakan lebih lanjut (restore atau repair). |

## 6 Cara Membuat Simulasi Fragmentasi (Untuk Praktikum)

Fragmentasi terjadi secara alami ketika data sering diinsert, update, delete secara acak. Untuk praktikum, kita memaksa fragmentasi dengan insert data acak:

### Insert Data Acak (Paksa Fragmentasi):

```
DECLARE @i INT = 1;
WHILE @i <= 3000
BEGIN
    INSERT INTO akademik.Mahasiswa
    VALUES (
        RIGHT('00000000000' + CAST(ABS(CHECKSUM(NEWID()))) % 9999999999 AS
    VARCHAR), 11),
        'RandomNama' + CAST(@i AS VARCHAR),
        GETDATE(),
        'TI',
        2024
    );
    SET @i = @i + 1;
END
```

### 💡 Mengapa Ini Menyebabkan Fragmentasi?

NIM adalah Clustered Index (data disimpan terurut berdasarkan NIM).

Saat kita insert NIM acak (NEWID()), SQL Server harus menyisipkan baris di TENGAH urutan yang sudah ada.

Ini menyebabkan 'page split' → halaman data terpecah-pecah → fragmentasi naik!

Ibarat: kamu punya buku terurut A-Z, lalu terus-terusan menyisipkan halaman di halaman M (tengah).

## 7 Alur Maintenance Lengkap — Ringkasan

| No | Langkah                      | Perintah/Cara                         |
|----|------------------------------|---------------------------------------|
| 1  | Cek struktur index           | SELECT dari sys.indexes               |
| 2  | Cek fragmentasi awal         | sys.dm_db_index_physical_stats        |
| 3  | Buat simulasi fragmentasi    | INSERT data acak                      |
| 4  | Cek fragmentasi lagi         | Pastikan sudah 5% atau lebih          |
| 5  | REORGANIZE (jika 5-30%)      | ALTER INDEX ALL ... REORGANIZE        |
| 6  | Cek fragmentasi turun        | Verifikasi hasilnya                   |
| 7  | REBUILD (jika > 30%)         | ALTER INDEX ALL ... REBUILD           |
| 8  | Cek fragmentasi mendekati 0% | Verifikasi hasilnya                   |
| 9  | UPDATE STATISTICS            | UPDATE STATISTICS atau sp_updatestats |
| 10 | DBCC CHECKDB                 | Pastikan database sehat               |

# CHEAT SHEET — PERINTAH PENTING

Kumpulan perintah SQL paling penting dari ketiga materi. Hafalkan ini!

## Backup & Restore (Materi 7 & 8)

```
-- Ubah ke FULL Recovery
ALTER DATABASE DBKampus SET RECOVERY FULL;

-- Full Backup
BACKUP DATABASE DBKampus TO DISK = 'D:\BackupSQL\DBKampus_Full.bak'
WITH FORMAT, INIT, NAME = 'Full Backup', STATS = 10;

-- Differential Backup
BACKUP DATABASE DBKampus TO DISK = 'D:\BackupSQL\DBKampus_Diff.bak'
WITH DIFFERENTIAL, INIT, NAME = 'Differential Backup', STATS = 10;

-- Transaction Log Backup
BACKUP LOG DBKampus TO DISK = 'D:\BackupSQL\DBKampus_Log1.trn'
WITH INIT, STATS = 10;

-- Restore Full (langkah 1/2, belum selesai)
RESTORE DATABASE DBKampus FROM DISK = 'D:\BackupSQL\DBKampus_Full.bak'
WITH NORECOVERY, REPLACE, STATS = 10;

-- Restore Differential (langkah 2/2, selesai)
RESTORE DATABASE DBKampus FROM DISK = 'D:\BackupSQL\DBKampus_Diff.bak'
WITH RECOVERY, STATS = 10;

-- Point-in-Time Recovery (Log dengan STOPAT)
RESTORE LOG DBKampus FROM DISK = 'D:\BackupSQL\DBKampus_Log2.trn'
WITH STOPAT = '2026-05-05 06:10:00', RECOVERY;

-- Single User Mode (sebelum restore)
USE master; ALTER DATABASE DBKampus SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

-- Multi User Mode (setelah restore)
ALTER DATABASE DBKampus SET MULTI_USER;
```

## Maintenance (Materi 9)

```
-- Cek fragmentasi
SELECT OBJECT_NAME(object_id) AS NamaTabel,
       avg_fragmentation_in_percent, page_count
FROM sys.dm_db_index_physical_stats
     (DB_ID(), OBJECT_ID('akademik.Mahasiswa'), NULL, NULL, NULL);

-- REORGANIZE (fragmentasi 5-30%)
ALTER INDEX ALL ON akademik.Mahasiswa REORGANIZE;

-- REBUILD (fragmentasi > 30%)
ALTER INDEX ALL ON akademik.Mahasiswa REBUILD;

-- Update statistics satu tabel
UPDATE STATISTICS akademik.Mahasiswa;
```

```
-- Update statistics seluruh database
EXEC sp_updatestats;

-- Cek integritas database
DBCC CHECKDB ('DBKampus') WITH NO_INFOMSGS;

-- Cek status database
SELECT name, state_desc FROM sys.databases WHERE name = 'DBKampus';

-- Monitor progress restore
SELECT percent_complete, start_time, command
FROM sys.dm_exec_requests WHERE command LIKE 'RESTORE%';
```

## Tabel Perbandingan Semua Konsep

| Operasi                  | Cakupan                  | Catatan                                | Kapan Dibutuhkan          | Frekuensi       |
|--------------------------|--------------------------|----------------------------------------|---------------------------|-----------------|
| <b>Full Backup</b>       | Seluruh DB               | NORECOVERY jika ada backup selanjutnya | Full Backup               | Setiap minggu   |
| <b>Differential</b>      | Sejak Full terakhir      | RECOVERY jika ini backup terakhir      | Full Backup               | Setiap hari     |
| <b>Transaction Log</b>   | Sejak Log terakhir       | STOPAT untuk point-in-time             | Full + Log chain          | Setiap jam      |
| <b>REORGANIZE</b>        | Fragmentasi 5-30%        | Online, tidak lock database            | Fragmentasi naik          | Seminggu sekali |
| <b>REBUILD</b>           | Fragmentasi > 30%        | Offline (lock database)                | Fragmentasi sangat tinggi | Sebulan sekali  |
| <b>UPDATE STATISTICS</b> | Setelah banyak perubahan | Satu tabel atau seluruh DB             | Data berubah banyak       | Setiap hari     |
| <b>DBCC CHECKDB</b>      | Seluruh database         | Hanya monitoring, tidak memperbaiki    | Rutin                     | Seminggu sekali |

### 🌸 TIPS UJIAN — Hal yang Sering Ditanyakan

1. Urutan restore: Full (NORECOVERY) → Differential/Log (RECOVERY). Jangan terbalik!
2. NORECOVERY = masih ada backup lagi yang mau di-apply. RECOVERY = ini yang terakhir, aktifkan DB.
3. STOPAT digunakan pada Log Backup TERAKHIR untuk point-in-time recovery.
4. REPLACE digunakan saat restore untuk menimpa database yang sudah ada.
5. Fragmentasi 5-30% → REORGANIZE. Fragmentasi >30% → REBUILD.
6. Setelah REBUILD, selalu jalankan UPDATE STATISTICS.
7. DBCC CHECKDB WITH NO\_INFOMSGS → hanya tampilkan error, output bersih jika sehat.
8. Recovery Model FULL wajib sebelum bisa Log Backup.
9. Setelah ubah ke FULL, wajib Full Backup dulu sebelum bisa Log Backup.

10. Log Backup membentuk 'rantai' → jika satu file .trn hilang, restore putus!